BÖLÜM 3 / MODULE 3

Önbellekleme Mimarisi ve CDN Stratejileri

Distributed Caching Architectures & CDN Strategies

Donanım gecikme piramidi, Cache-Aside vs Write-Through, LRU tahliye mekanizması, Redis Cluster, Cache Stampede/Avalanche felaketleri ve Kenar CDN mimarisi.

Hardware latency hierarchy, Cache-Aside vs Write-Through, LRU evictions, Redis clustering, Thundering Herd mitigations, and Edge CDN deployment topologies.

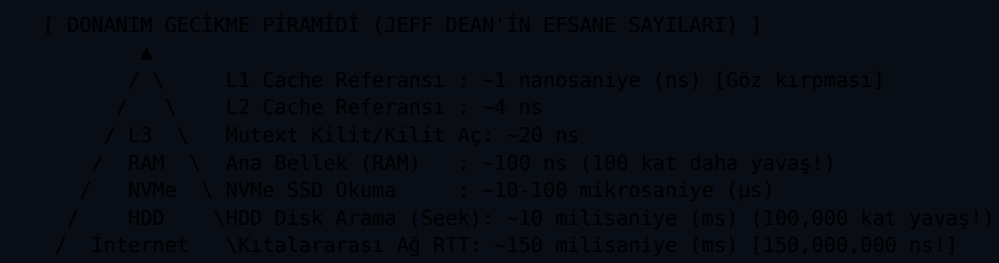
13

BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Önbellek Seviyeleri ve Donanım Piramidi (Her Mühendisin Bilmesi Gereken Sayılar)

Memory Hierarchy & Latency Numbers Every Engineer Must Know

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Donanım hızlarını insan boyutuna uyarlayalım: L1 önbellekten veri okumak, masanızın üzerindeki kağıda bakmak gibidir (1 saniye sürer). RAM'den okumak, odadan çıkıp alt kattaki arşiv odasına yürümektir (1.5 dakika sürer). NVMe SSD'den okumak, komşu şehre arabayla gidip evrak almaktır (1.5 gün sürer). Dönen mekanik hard diskten (HDD) okumak, 3 ay süren bir dünya turuna çıkmaktır! İnternet üzerinden başka bir ülkeden veri çekmek ise Mars'a gidip gelmek gibidir (5 yıl sürer)! İşte önbellek, veriyi Mars'tan veya arşivden alıp masanızın üzerine koyma sanatıdır!

🇬🇧 ENGLISH (ELI5)

Scaling computer latencies to human time: An L1 CPU cache reference is glancing at a sticky note on your desk (1 second). Fetching from RAM is walking downstairs to the building archives (90 seconds). Reading from an NVMe SSD is driving to a neighboring city (1.5 days). Reading from a spinning magnetic HDD is boarding a sailboat for a 3-month trans-Atlantic voyage! Fetching from a cross-continental remote API is a 5-year round trip to Mars! Caching is the supreme discipline of keeping hot data glued directly to your desk!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Jeff Dean'in ünlü 'Numbers Every Programmer Should Know' tablosu sistem tasarımının fiziksel zeminidir: L1 Cache: 0.5-1 ns, L2: 3-4 ns, RAM: 100 ns, SSD rastgele okuma: 16-100 µs, Veri merkezi içi RTT: 500 µs, HDD arama: 10 ms, Kaliforniya-Hollanda RTT: 150 ms. Bir veritabanı sorgusu disk ve ağ IO içerdiğinde milisaniyeler sürerken; RAM'deki bir Redis önbelleği mikrosaniyeler mertebesindedir (1.000 kat daha hızlı). Mülakatta veritabanı üzerindeki okuma yükünü %90 azaltmanın tek yolu bu piramidi doğru katmanlamaktır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Jeff Dean's latency benchmarks dictate distributed system sizing: L1: 1ns, L2: 4ns, RAM: 100ns, NVMe SSD read: 20µs, Datacenter RTT: 0.5ms, Mechanical disk seek: 10ms, Trans-Atlantic RTT: 150ms. Disk operations are orders of magnitude slower than DRAM access. Placing an in-memory Redis or Caffeine layer in front of relational databases reduces latency percentiles from 15ms down to 500µs while absorbing 95% of read pressure.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Donanım Gecikme Skalasını İnsan Zamanına Çeviren Model
public record LatencyScale(String operation, long nanos) {
    // 1 nanosaniyeyi 1 saniye kabul ederek insan ölçeğine çevir
    public String toHumanScale() {
        long seconds = nanos;
        if (seconds < 60) return seconds + " saniye";
        long minutes = seconds / 60;
        if (minutes < 60) return minutes + " dakika";
        long hours = minutes / 60;
        if (hours < 24) return hours + " saat";
        long days = hours / 24;
        if (days < 365) return days + " gün";
        return (days / 365) + " yıl";
    }
}

void main() {
    var l1 = new LatencyScale("L1 Cache", 1L);
    var ram = new LatencyScale("RAM Bellek", 100L);
    var ssd = new LatencyScale("NVMe SSD", 100_000L);
    var net = new LatencyScale("Trans-Atlantik Ağ", 150_000_000L);

    IO.println(l1.operation() + " -> " + l1.toHumanScale());
    IO.println(ram.operation() + " -> " + ram.toHumanScale());
    IO.println(ssd.operation() + " -> " + ssd.toHumanScale());
    IO.println(net.operation() + " -> " + net.toHumanScale());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'RAM ucuz, her şeyi sonsuza kadar Redis'te saklarım' demeyin! RAM, SSD'ye göre gigabayt başına yaklaşık 5-10 kat daha pahalıdır. Ön bellek sadece 'Sıcak Veri' (Hot Working Set) içindir; soğuk veriler daima disk/nesne depolamada tutulmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never declare 'RAM is cheap, I'll cache the entire 100TB database in Redis'. DRAM is 5x to 10x more expensive per gigabyte than NVMe. Caches are exclusively reserved for the 20% hot active working set.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- RAM diskten 1.000 kat, ağ gecikmesinden 1.000.000 kat daha hızlıdır.
- Sistem performansının en büyük düşmanı disk I/O ve ağ RTT gecikmesidir.
- Önbellek, sıcak veriyi donanım piramidinin en tepesine yaklaştırma stratejisidir.

🎯 Key Takeaways for the Interview

- DRAM is 1,000x faster than NVMe disks and 1,000,000x faster than cross-regional networks.
- The primary latency bottlenecks in web services are disk IO seeks and network RTT.
- Caching bridges physical latency penalties by lifting hot data to the top of the pyramid.

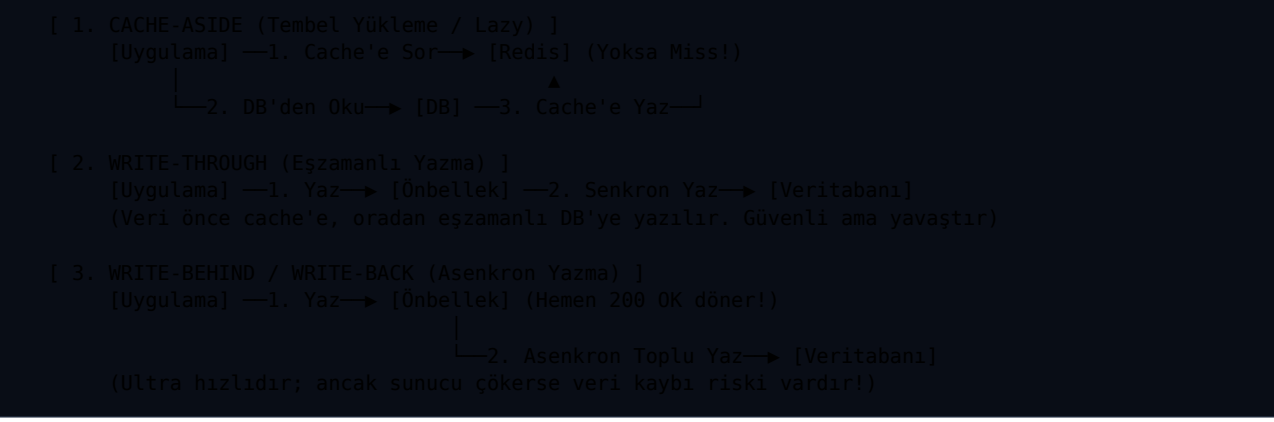
14

BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Önbellek Yazma Stratejileri: Cache-Aside, Write-Through ve Write-Back

Cache Writing Patterns: Cache-Aside, Write-Through & Write-Behind

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir not defteriniz (Cache) ve kalın bir arşiv defteriniz (DB) olsun: 1) Cache-Aside: Bir soru sorulduğunda önce not defterine bakarsınız; yoksa arşivden bulup not defterinize iliştirirsiniz. 2) Write-Through: Yeni bir bilgi geldiğinde aynı anda hem not defterine hem arşive yazıp öyle kafanızı kaldırırsınız (ikisi de hep günceldir ama yavaş yazarsınız). 3) Write-Back: Yeni bilgiyi hemen not defterine karalar ve 'Tamam!' dersiniz; akşama doğru arşiv defterine topluca geçirirsiniz. Ultra hızlıdır ama not defterini çay dökülüp kaybolursa bilgiler silinir!

🇬🇧 ENGLISH (ELI5)

Imagine a pocket notepad (Cache) and a heavy leather ledger (Database): 1) Cache-Aside: When asked a question, check your notepad; on a miss, look up the ledger and scribble the answer into your notepad for next time. 2) Write-Through: When recording an entry, write simultaneously to the notepad AND the ledger before responding (bulletproof consistency, higher write latency). 3) Write-Behind (Write-Back): Scribble into the notepad instantly and confirm completion; asynchronously batch-sync to the ledger later (extreme write speed, but if the notepad burns before sync, data vaporizes)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) Cache-Aside (Lazy Loading): En yaygın modeldir (Memcached, Redis). Uygulama kodu önbellek ve veritabanı koordinasyonunu bizzat yönetir. Sadece talep edilen veriler önbelleğe alınır; dezavantajı: Cache Miss durumunda 3 ağ gidiş-gelişi (Cache oku -> DB oku -> Cache yaz). Veri güncellendiğinde DB yazılır ve cache `EVICT` (silinir) edilir. 2) Write-Through: Önbellek bir ara katmandır. Veri önce cache'e yazılır, cache senkron olarak DB'ye yazar. Tutarlılık yüksektir ancak yazma gecikmesi artar. 3) Write-Behind / Write-Back: Veri cache'e yazılır ve anında yanıt dönülür; arka plan worker'ları veriyi kuyruktan alıp DB'ye asenkron toplu (batch) yazar. Yoğun yazma trafiği olan sistemlerde (oyun skorları, sayaçlar, IoT sensör verileri) kullanılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) Cache-Aside: The application directly orchestrates cache and persistence. Misses fetch from DB and populate the cache. Updates write to DB and evict stale cache keys. 2) Write-Through: The cache acts as the main persistence abstraction. Writes update cache first, which synchronously flushes to DB before returning, guaranteeing strict read consistency at the penalty of elevated write latency. 3) Write-Behind (Write-Back): Writes hit the in-memory cache and return immediately (sub-millisecond ACK); background workers batch-drain dirty writes to DB asynchronously. Delivers unmatched write throughput for metrics, telemetry, and live view counters at the risk of transient data loss on node crashes.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Cache-Aside & Write-Behind Simülasyonu
import java.util.concurrent.*;

public class CachePatterns {
    private final ConcurrentMap<String, String> cache = new ConcurrentHashMap<>();
    private final BlockingQueue<String> writeBehindQueue = new LinkedBlockingQueue<>();

    // 1. Cache-Aside Okuma Mantığı
    public String getWithCacheAside(String key) {
        // Cache kontrolü
        String cached = cache.get(key);
        if (cached != null) return cached;

        // DB simülasyonu (Cache Miss)
        String dbValue = "DB_VAL_FOR_" + key;
        cache.put(key, dbValue);
        return dbValue;
    }

    // 2. Write-Behind (Asenkron Kuyruğa Atma)
    public void writeBehind(String key, String value) {
        cache.put(key, value); // RAM'e hemen yaz
        writeBehindQueue.offer(key + "=" + value); // DB worker'ına havale et
    }
}

void main() {
    var patterns = new CachePatterns();
    IO.println("Cache-Aside Değeri: " + patterns.getWithCacheAside("user:101"));
    patterns.writeBehind("score:99", "1500");
    IO.println("Write-Behind Başarılı (Bellek Güncellendi)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Cache-Aside deseninde güncelleme yaparken önbelleğe yeni değeri YAZMAYIN (Cache Update)! Eski değeri SİLİN (Cache Eviction / Invalidation)! İki eşzamanlı yazma işlemi yarıştığında (Race Condition), önbellekte bayat veri kalabilir. En güvenli kural: DB'ye yaz, cache'i sil!

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

During mutations in Cache-Aside, do NOT update the cache in-place! DELETE/EVICT the key instead! In concurrent write races, in-place updates leave stale state. The golden rule: Commit to DB, then EVICT the cache key.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Cache-Aside okuma ağırlıklı (read-heavy) sistemlerin tartışmasız standart desenidir.
- Veri güncellendiğinde önbelleği güncellemek yerine anahtarı silmek (evict) yarış durumlarını önler.
- Write-Behind devasa yazma hacimlerini (yüksek QPS) veritabanına toplu aktarmak için idealdir.

🎯 Key Takeaways for the Interview

- Cache-Aside is the default pattern for read-heavy distributed architectures.
- Evicting cache keys upon DB mutations avoids dangerous concurrency race conditions.
- Write-Behind consolidates massive write bursts into efficient batch database flushes.

15

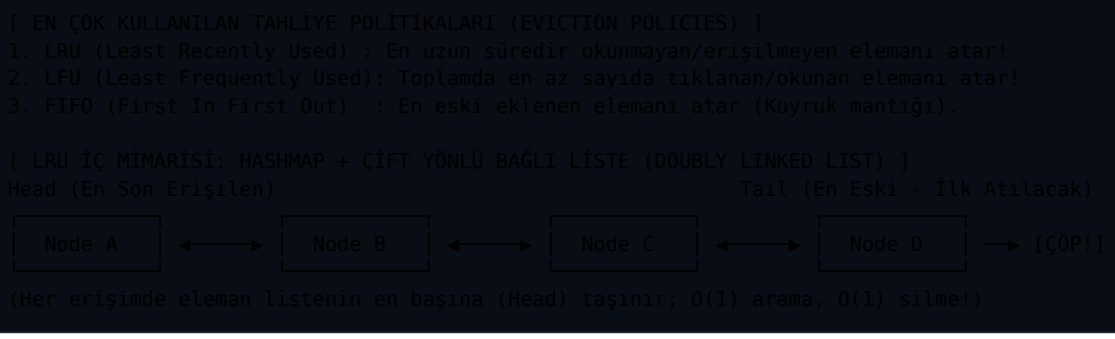
BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Önbellek Tahliye Algoritmaları: LRU, LFU, FIFO ve Java 25 LRU Cache

Cache Eviction Algorithms: LRU, LFU, FIFO & Java 25 LRU Cache

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Çalışma masanızın üzerinde en fazla 5 kitap koyabileceğiniz bir alan olduğunu hayal edin (Önbellek boyutu). 6. kitabı koymak için birini rafa kaldırmalısınız. LRU (Least Recently Used), 'En uzun süredir kapağını bile açmadığım kitabı kaldırayım' diyen mantıktır. LFU (Least Frequently Used) ise 'Sene boyunca toplamda en az okuduğum kitabı kaldırayım' der. Masadaki kitaplardan birini her okuduğunuzda onu masanın en önüne (Head) çekersiniz; masanın en arkasında kalan kitap (Tail) ise ilk atılacak kurbandır!

🇬🇧 ENGLISH (ELI5)

Imagine your desk only fits 5 textbooks. To place a 6th book, you must return one to the library shelf. LRU (Least Recently Used) evicts the book you haven't touched in the longest time. LFU (Least Frequently Used) counts tally marks and evicts the book with the lowest total read count. Whenever you consult a book, you slide it to the front of your desk. The book lingering neglected at the far corner is the first victim evicted on overflow!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG mülakatlarında en çok kodlatılan veri yapısı LRU Cache'tir. Bir LRU Cache iki temel işlem için $O(1)$ zaman karmaşıklığı sunmalıdır: 1) `get(key)`: Anahtarı bulmalı ve düğümü en başa (Head) taşımalıdır. 2) `put(key, value)`: Yeni düğümü başa eklemeli, kapasite aşıldığında sondaki (Tail) düğümü silmelidir. Bu $O(1)$ garantisi HashMap ($O(1)$ anahtar araması) ile Doubly Linked List'in ($O(1)$ düğüm çıkarma ve ekleme) birleşimiyle sağlanır. LFU ise frekans sayacı tutar; ancak ani popülerlik dalgalanmalarına (flash crowds) uyum sağlaması zordur.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The LRU Cache is a premier FAANG coding interview staple. It guarantees $O(1)$ time complexity for both `get` and `put` operations. This is engineered by coupling a Hash Table (for $O(1)$ pointer lookups) with a Doubly Linked List (for $O(1)$ node extractions and insertions at Head and Tail boundaries). When a node is touched, it splices out of its current position and prepends to the Head. On capacity overflow, the Tail node drops. LFU maintains frequency buckets but suffers from historical inertia where old items refuse to evict despite going cold.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: O(1) Zaman Karmaşıklığına Sahip Gerçek LRU Cache Implementasyonu
import java.util.*;

public class LruCache<K, V> {
    private static class Node<K, V> {
        K key; V value;
        Node<K, V> prev, next;
        Node(K k, V v) { this.key = k; this.value = v; }
    }

    private final int capacity;
    private final Map<K, Node<K, V>> map = new HashMap<>();
    private final Node<K, V> head = new Node<>(null, null);
    private final Node<K, V> tail = new Node<>(null, null);

    public LruCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public synchronized V get(K key) {
        Node<K, V> node = map.get(key);
        if (node == null) return null;
        moveToHead(node);
        return node.value;
    }

    public synchronized void put(K key, V value) {
        Node<K, V> node = map.get(key);
        if (node != null) {
            node.value = value;
            moveToHead(node);
        } else {
            if (map.size() >= capacity) {
                map.remove(tail.prev.key);
                removeNode(tail.prev);
            }
            Node<K, V> newNode = new Node<>(key, value);
            map.put(key, newNode);
            addToHead(newNode);
        }
    }

    private void removeNode(Node<K, V> node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void addToHead(Node<K, V> node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }

    private void moveToHead(Node<K, V> node) {
        removeNode(node);
        addToHead(node);
    }
}

void main() {
    var lru = new LruCache<String, Integer>(2);
    lru.put("A", 1);
    lru.put("B", 2);
    IO.println("Get A: " + lru.get("A")); // A başa geçti, B arkaya düştü
    lru.put("C", 3);                      // Kapasite taştı -> B atıldı!
    IO.println("Get B (Tahliye Edildi): " + lru.get("B")); // null döner!
    IO.println("Get C: " + lru.get("C"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Java'nın `LinkedHashMap(capacity, 0.75f, true)` sınıfı LRU mantığını hazır sunar. Ancak MAANG mülakatında hazır kütüphane çağırmak yerine HashMap + Doubly Linked List veri yapısını kendi elinizle çizip açıklamanız beklenir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

While Java provides `LinkedHashMap(..., accessOrder=true)`, invoking it without explaining the underlying Doubly Linked List and Hash Map duality will disqualify your algorithmic rigor in Staff rounds.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- LRU en son erişilen veriyi korur ve zamansal yerellik (temporal locality) için mükemmeldir.
- O(1) süre garantisi HashMap ile Çift Yönlü Bağlı Liste kombinasyonuyla elde edilir.
- Redis varsayılan olarak yaklaşık (approximate) LRU algoritması kullanarak bellekten tasarruf eder.

🎯 Key Takeaways for the Interview

- LRU leverages temporal locality, keeping recently accessed objects warm.
- Combining a Hash Map with a Doubly Linked List yields strict O(1) lookups and evictions.
- Production Redis uses probabilistic sampling to approximate LRU with zero pointer overhead.

16

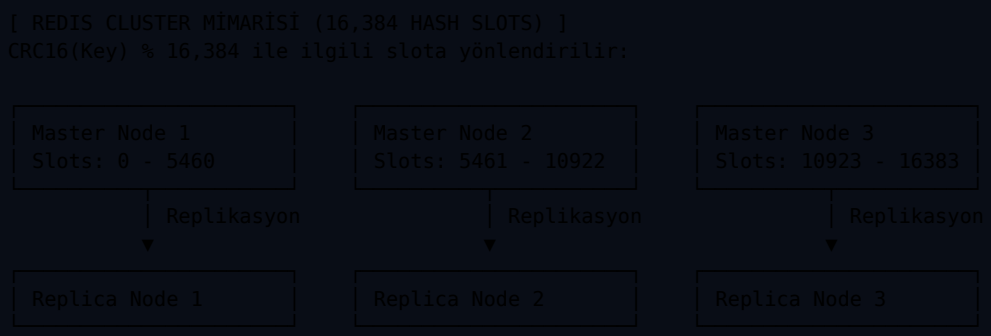
BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Dağıtık Önbellek: Redis ve Memcached Mimarisi (Sentinel vs Cluster)

Distributed Cache: Redis vs Memcached (Sentinel & Cluster Sharding)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Redis ile Memcached iki ünlü yarış arabası gibidir: Memcached sadece dümdüz ileri giden, çok hafif ve basit bir drag yarışçısıdır (Sadece Key-Value tutar, çok çekirdek kullanır). Redis ise İsviçre çakısı gibi donatılmış bir Formula 1 aracıdır! Listeler, Kümeler (Sets), Sıralı Kümeler (Sorted Sets), Bitmap'ler, Pub/Sub ve diske kaydetme (Persistence) yetenekleri vardır. Redis Cluster ise bu Formula 1 filosunun 3 farklı pistte (16,384 yuva - hash slot) yarışarak yükü paylaşmasıdır!

🇬🇧 ENGLISH (ELI5)

Redis and Memcached are two performance sports cars: Memcached is a stripped-down dragster built for raw multi-threaded speed (pure Key-Value strings). Redis is a hypercar engineered with an entire Swiss Army knife! It natively manipulates Hashes, Lists, Sets, Sorted Sets (ZSET for leaderboards), Bitmaps, HyperLogLogs, and Pub/Sub streams with disk durability (RDB/AOF). Redis Cluster partitions keys across 16,384 distributed hash slots for horizontal scaling!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Memcached: Multi-threaded mimariye sahiptir; çok çekirdekli makinelerin CPU gücünü tam kullanır; veri tipleri yoktur (sadece binary string); disk kalıcılığı yoktur. Redis: Çekirdek motoru tek iş parçacıklıdır (Single-threaded event loop), bu sayede race condition ve kilit (lock) kargaşası olmadan komutları atomik yürütür (Network IO ise multithreaded'dir). Redis Mimarileri: 1) Redis Sentinel: Yüksek erişilebilirlik (HA) sağlar; Master çökerse otomatik Replica'yı Master yapar (Failover). 2) Redis Cluster: Yatay ölçekleme (Sharding) sağlar. Anahtarlar `CRC16(key) % 16384` formülüyle 16,384 Hash Slot'a bölünür ve master düğümler arasında paylaştırılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Memcached utilizes a multithreaded architecture maximizing multi-core symmetric multiprocessing, but supports only basic string key-values without disk durability. Redis runs an ultra-fast non-blocking single-threaded event loop, guaranteeing atomic executions across rich data structures (Lists, ZSETs, Streams, Hashes) without mutex locking overhead. Scalability options: 1) Redis Sentinel provides automatic master-replica failover monitoring. 2) Redis Cluster delivers true horizontal sharding: keys map to 16,384 hash slots via `CRC16(key) % 16384`, distributing partitions across autonomous master-replica shards with zero single points of failure.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Redis Hash Slot Hesaplayıcı Simülasyonu (CRC16 % 16384)
public class RedisClusterSlotRouter {
    private static final int TOTAL_SLOTS = 16384;

    // Basitleştirilmiş Hash Slot Hesaplama
    public static int calculateSlot(String key) {
        // Redis Hash Tag Desteği: {user:101}.orders -> Sadece {} içi hashlenir!
        int start = key.indexOf('{');
        int end = key.indexOf('}');
        String hashKey = (start != -1 && end > start) ? key.substring(start + 1, end) : key;

        return Math.abs(hashKey.hashCode() % TOTAL_SLOTS);
    }
}

void main() {
    String key1 = "product:iphone16";
    String key2 = "user:9941";
    String key3 = "{tenant:acme}.orders";
    String key4 = "{tenant:acme}.invoices"; // key3 ile KESİNLİKLE aynı slot'a gider!

    IO.println(key1 + " -> Slot #" + RedisClusterSlotRouter.calculateSlot(key1));
    IO.println(key2 + " -> Slot #" + RedisClusterSlotRouter.calculateSlot(key2));
    IO.println(key3 + " -> Slot #" + RedisClusterSlotRouter.calculateSlot(key3));
    IO.println(key4 + " -> Slot #" + RedisClusterSlotRouter.calculateSlot(key4));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Redis Cluster'da çoklu anahtar (MGET, MSET veya Transactions) çalıştırırken anahtarlar farklı slotlardaysa 'CROSSSLOT Keys in request don't hash to the same slot' hatası patlar! İlişkili verilerin aynı slota düşmesini sağlamak için Hash Tag `\{tenant_id\}.key` sözdizimini kullanın.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Redis Cluster throws CROSSSLOT errors when batching multi-key commands across different hash slots. Enforce colocation using Hash Tags (`\{user_id\}.profile` and `\{user_id\}.orders`) so related keys evaluate to the exact same shard.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Redis zengin veri tipleri ve atomik işlemleriyle modern sistemlerin vazgeçilmezidir.
- Redis Cluster, 16,384 Hash Slot yapısıyla veriyi onlarca sunucuya şeffafça böler.
- Hash Tags (\{tag\}) tek işlemde birden fazla anahtarı sorgulamak için slot sabitleme sağlar.

🎯 Key Takeaways for the Interview

- Redis is the premier caching engine due to its rich native data types and atomic primitives.
- Redis Cluster transparently shards datasets across 16,384 deterministic hash slots.
- Hash Tags ensure co-partitioning for multi-key transactional pipelines on clustered shards.

17

BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Önbellek Felaketleri: Cache Avalanche, Stampede (Thundering Herd) ve Penetration

Cache Catastrophes: Avalanche, Stampede (Thundering Herd) & Penetration

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram

```
[ 1. CACHE AVALANCHE (ÇIĞ ETKİSİ) ]
Milyonlarca anahtarın TTL'i aynı anda (ör. gece 00:00) biter!
Önbellek ──► %100 Boşalır ──► Milyonlarca İstek Veritabanına Hücum Eder ──► DB ÇÖKER!
Çözüm: TTL değerine rastgele Jitter (± 5 dakika) ekleyin!

[ 2. CACHE STAMPEDE / THUNDERING HERD (SÜRÜ İZDIHAMI) ]
Çok popüler tek bir anahtarın (Messi'nin profili) TTL'i biter.
Saniyede 50.000 istek aynı anda DB'ye koşup o tek satırı hesaplamaya çalışır!
Çözüm: Dağıtık Kilit (Mutex / Redlock) veya Arka Planda Proaktif Yenileme!

[ 3. CACHE PENETRATION (DELME SALDIRISI) ]
Saldırgan DB'de OLMAYAN rastgele ID'ler ister (ID: -9999999).
Cache Miss ──► DB sorgusu (Bulunamadı) ──► Cache'e yazılmaz ──► Saldırı DB'yi boğar!
Çözüm: Bloom Filter veya Boş Sonucu (Null Value) kısa TTL ile önbelleğe alma!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bu üç felaketi lokantada anlayalım: 1) Çığ Etkisi (Avalanche): Mutfaktaki tüm hazır yemeklerin aynı anda saat 12:00'de bozulup çöpe atılmasıdır; kapıdaki 500 müşteri doğrudan ocağın başına üşüşür ve mutfak yanar! 2) Sürü İzdihamı (Stampede): Menüdeki en sevilen tatlı biter; 1.000 müşteri aynı anda garsona 'Hemen aşçıya söyle bu tatlıyı yapsın' der, aşçı 1.000 kez aynı tatlıyı pişirmeye çalışırken bayılır! 3) Delme Saldırısı (Penetration): Kötü niyetli biri garsona 'Menüde olmayan Ejderha Çorbası var mı?' diye 100.000 kez sorar. Garson her seferinde depoya gidip bakar, depoyu kilitler!

🇬🇧 ENGLISH (ELI5)

Three restaurant nightmares: 1) Cache Avalanche: Every prepared buffet dish expires at precisely 12:00 PM; a swarm of 500 hungry diners surges straight into the kitchen, suffocating the stoves! 2) Cache Stampede (Thundering Herd): The signature chocolate cake runs out; 10,000 diners concurrently order the chef to bake a new cake, suffocating the pastry station! 3) Cache Penetration: A malicious troll repeatedly asks: 'Do you have Dragon Steak?' (an item that doesn't exist). The waiter marches into the basement warehouse 50,000 times, grinding service to a halt!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) Cache Avalanche: Yüzbinlerce önbellek anahtarının aynı TTL ile süresinin dolması sonucu isteklerin doğrudan veritabanını çökertmesidir. Çözüm: TTL süresine rastgele gürültü (Jitter) eklemek (`TTL = 3600 + rand(0, 300)`). 2) Cache Stampede: Yüksek QPS'li tek bir sıcak anahtarın süresi dolduğunda binlerce eşzamanlı thread'in DB'ye hücum etmesidir. Çözüm: Dağıtık Kilit (Mutex): Sadece ilk thread DB'ye gider, diğerleri bekler veya proaktif XFetch algoritması. 3) Cache Penetration: Veritabanında bulunmayan anahtarların sorgulanmasıyla cache'in delinip DB'nin boğulmasıdır. Çözüm: A) Bloom Filter: Olasılıksal veri yapısıyla elemanın DB'de kesinlikle olmadığını $O(1)$ sürede bellekten söyleyerek isteği reddeder. B) Null Caching: Boş sonucu kısa bir TTL (60 sn) ile önbelleğe almak.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) Cache Avalanche: Mass simultaneous TTL expirations collapse the database under an instantaneous traffic waterfall. Mitigation: Inject stochastic TTL jitter (`ttl = base_ttl + random_jitter`). 2) Cache Stampede (Thundering Herd): A hyper-popular hot key expires, triggering thousands of concurrent cache misses that hammer the database to compute the identical payload. Mitigation: Mutex locking (only one thread computes, others poll) or probabilistic early expiration (XFetch algorithm). 3) Cache Penetration: Adversaries query non-existent keys (e.g., negative UUIDs), bypassing cache and degrading backend disk IO. Mitigation: Deploy a space-efficient Bloom Filter at the gateway to reject non-existent keys in O(1) time, or cache `NULL` payloads with transient TTLs.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Cache Stampede (Mutex Kilit) & Avalanche (Jitter) Çözümleri
import java.util.concurrent.*;
import java.util.concurrent.locks.ReentrantLock;

public class CacheDefenses {
    private final ConcurrentMap<String, String> cache = new ConcurrentHashMap<>();
    private final ConcurrentMap<String, ReentrantLock> locks = new ConcurrentHashMap<>();

    // 1. Avalanche Koruması: Rastgele Jitter ile TTL Üretimi
    public int calculateTtlWithJitter(int baseTtlSeconds, int jitterRangeSeconds) {
        int jitter = ThreadLocalRandom.current().nextInt(jitterRangeSeconds);
        return baseTtlSeconds + jitter; // Örn: 3600 + [0-300] sn
    }

    // 2. Stampede Koruması: Mutex Kilit ile Tekil DB Sorgusu
    public String getWithMutex(String key) {
        String val = cache.get(key);
        if (val != null) return val;

        ReentrantLock lock = locks.computeIfAbsent(key, k -> new ReentrantLock());
        lock.lock();
        try {
            // Çift Kontrol (Double-Checked Locking)
            val = cache.get(key);
            if (val != null) return val;

            // Yalnızca TEK bir thread DB'ye gider!
            val = "HEAVY_DB_RESULT_FOR_" + key;
            cache.put(key, val);
            return val;
        } finally {
            lock.unlock();
        }
    }
}

void main() {
    var defense = new CacheDefenses();
    IO.println("Jitter'lı TTL: " + defense.calculateTtlWithJitter(3600, 300) + " saniye");
    IO.println("Stampede Korumalı Sonuç: " + defense.getWithMutex("hot:product:101"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Bloom Filter her şeyi kesin bilir' demeyin! Bloom Filter'ın doğasında 'False Positive' (aslında yokken var diyebilme) payı vardır; ancak 'False Negative' (varken yok deme) riski sıfırdır. Bu ayrımı vurgulamak mülakatçıyı büyüler.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never claim Bloom Filters are 100% accurate. Bloom Filters exhibit false positives (may report an item exists when it doesn't), but zero false negatives. Articulating this trade-off marks architectural mastery.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• TTL sürelerine mutlaka rastgele Jitter ekleyerek Çığ Etkisini (Avalanche) önleyin.

• Sıcak anahtarlar için Mutex kilidi kullanarak Sürü İzdihamını (Stampede) engelleyin.

• Bloom Filter ve Null Caching ile veritabanını Delme Saldırılarına (Penetration) karşı koruyun.

🎯 Key Takeaways for the Interview

• Inject stochastic jitter to key TTLs to eliminate concurrent cache avalanche cascades.

• Use distributed mutex locks to ensure only one thread recomputes a hot expired key.

• Shield databases from penetration attacks using in-memory Bloom Filters and Null Caching.

18

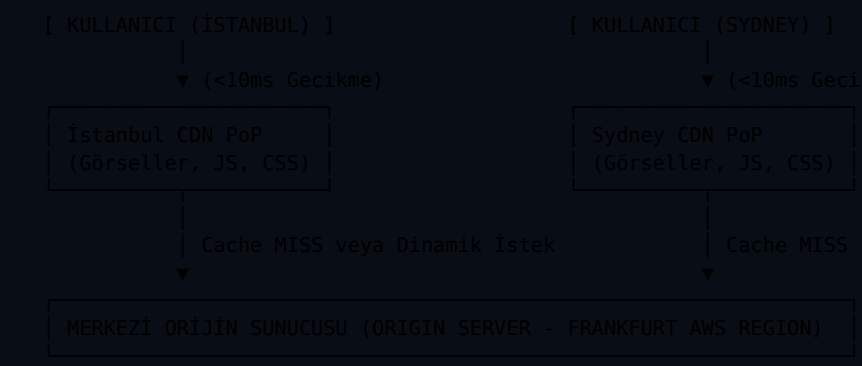
BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

CDN (Content Delivery Network) ve Kenar Hesaplama (Edge Computing & PoP)

CDNs & Edge Computing: PoPs, Anycast & Edge Workers

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Netflix'teki 4K bir filmi izlemek için her seferinde Kaliforniya'daki ana sunucuya bağlanıp okyanusun altındaki kabloları ısıtmak zorunda kalsaydınız film donardı! Bunun yerine Netflix, o filmin kopyalarını mahallenizdeki internet servis sağlayıcısının (ISP) içine yerleştirilmiş özel kutulara (CDN PoP - Point of Presence) koyar. Siz filmi izlerken Kaliforniya'dan değil, 5 kilometre ötenizdeki kutudan çekersiniz. CDN işte bu küresel film ve fotoğraf kütüphanesidir!

🇬🇧 ENGLISH (ELI5)

If streaming a 4K movie on Netflix required pulling bytes directly from an origin database in Los Angeles over trans-oceanic cables, video would buffer endlessly! Instead, Netflix pre-stages encrypted copies of popular media into custom appliances (Open Connect CDN PoPs) embedded directly inside your city's local ISP exchange. You stream from 5 miles away, not 7,000 miles across the ocean! That is a Content Delivery Network (CDN)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CDN (İçerik Dağıtım Ağı), dünya genelinde yüzlerce Point of Presence (PoP) noktasında çalışan ters proxy sunucuları ağıdır. 1) Statik İçerik Hızlandırma: Resimler, videolar, JS/CSS dosyaları origin'e gitmeden kenarda sunulur. Origin bant genişliği tüketimini %80-%95 azaltır. 2) Dinamik İçerik Hızlandırma: Dinamik API istekleri önbelleğe alınamasa bile, istemci ile PoP arasındaki TCP/TLS el sıkışması yerel tamamlanır ve origin ile PoP arasında açık tutulan optimize BGP tünelleri (Route Optimization) üzerinden akıtılır. 3) Edge Computing (Cloudflare Workers, AWS Lambda@Edge): JWT doğrulama, A/B testi, bot koruması ve coğrafi yönlendirme kodları doğrudan kenar sunucularda (Edge) çalıştırılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

A Content Delivery Network (CDN) is a globally distributed mesh of caching proxies deployed across hundreds of edge Points of Presence (PoPs). 1) Static Acceleration: Serves media, images, CSS, and binaries locally, offloading 90%+ of origin egress costs. 2) Dynamic Site Acceleration (DSA): Even uncacheable dynamic API payloads accelerate via early TLS handshakes at the edge PoP, traversing persistent, route-optimized BGP backbones to origin. 3) Edge Compute (Cloudflare Workers, Fastly Compute): Executes lightweight serverless logic (JWT validation, A/B routing, geolocation geofencing, bot mitigation) directly at edge nodes within 5ms of clients.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: CDN Cache-Control Başlık Üreticisi ve Edge Karar Motoru
public record CdnPolicy(int maxAgeSeconds, int sharedMaxAgeSeconds, boolean isPublic) {
    public String toHeaderValue() {
        return (isPublic ? "public, " : "private, ")
            + "max-age=" + maxAgeSeconds
            + ", s-maxage=" + sharedMaxAgeSeconds // s-maxage CDN için geçerlidir!
            + ", stale-while-revalidate=60";
    }
}

void main() {
    // Statik Profil Resmi Politikası: Tarayıcı 1 saat, CDN 24 saat önbelleğe alsın
    var avatarPolicy = new CdnPolicy(3600, 86400, true);
    IO.println("Cache-Control Başlığı: " + avatarPolicy.toHeaderValue());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'CDN sadece statik resimler içindir' demeyin! Modern CDN'ler (Cloudflare, Fastly, CloudFront); WAF (Web Application Firewall), DDoS kalkanı, TLS sonlandırma, dinamik rota hızlandırma ve Edge Compute (Sunucusuz kod çalıştırma) yapar.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never dismiss CDNs as simple image repositories. Modern edge networks handle DDoS scrubbing, WAF inspection, TLS termination, dynamic route optimization, and edge compute execution.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• CDN statik içerikleri kullanıcıya en yakın PoP noktasından milisaniyeler içinde sunar.

• Origin sunucusuna binen bant genişliği ve sunucu maliyetini %90'a varan oranda düşürür.

• Edge Workers ile kimlik kontrolü ve yönlendirme mantığı istemciye en yakın noktada yürütülür.

🎯 Key Takeaways for the Interview

• CDNs deliver static assets from local PoPs with sub-10ms round-trip latencies.

• Edge caching shields origin infrastructures, cutting egress transit bills by up to 90%.

• Edge computing offloads perimeter authentication and traffic routing directly to the network fringe.